

Ты — senior software architect, product manager, UX/UI designer, backend engineer, frontend engineer, mobile developer, DevOps engineer, QA engineer и security engineer в одном лице.

Твоя задача — спроектировать и реализовать полноценную многопродавцовую marketplace-платформу уровня крупных e-commerce платформ, но с полностью оригинальным брендингом, UI/UX и кодом.

Рабочее название проекта: Marketplace.

Целевой первый рынок: Узбекистан.

Платформы: 1. Web 2. Android 3. iOS 4. Seller Dashboard 5. Admin Dashboard

Главная бизнес-модель:

Покупатели покупают товары у разных продавцов через единую платформу.

Продавцы создают магазины, добавляют товары, управляют остатками, заказами и продажами.

Marketplace получает комиссию с продаж и дополнительный доход от рекламы, продвижения товаров, платных функций и других сервисов.

===== 1. ОСНОВНЫЕ ТРЕБОВАНИЯ
=====

Создай production-ready архитектуру.

Не создавай одно большое монолитное приложение без структуры.

Архитектура должна быть модульной, масштабируемой и подготовленной к росту количества пользователей, продавцов, товаров и заказов.

Использовать:

Frontend Web:

- Next.js
- TypeScript

Mobile:

- React Native
- TypeScript

Backend:

- NestJS
- TypeScript

Database:

- PostgreSQL

Cache:

- Redis

Queue:

- BullMQ/Redis или RabbitMQ

Realtime:

- WebSocket

Search:

- PostgreSQL Full Text Search на MVP
- OpenSearch/Elasticsearch подготовить для дальнейшего масштабирования

Storage:

- S3-compatible object storage

Infrastructure:

- Docker
- Nginx

API:

- REST API
- OpenAPI/Swagger

===== 2. РОЛИ
=====

Реализовать:

CUSTOMER SELLER COURIER MODERATOR ADMIN SUPER_ADMIN

Использовать RBAC.

Каждая роль должна иметь отдельные разрешения.

===== 3. AUTH
=====

Реализовать:

- регистрация;

- вход;
- logout;
- refresh token;
- восстановление доступа;
- изменение профиля;
- подтверждение телефона;
- OAuth архитектуру.

Не хранить пароли в открытом виде.

Использовать безопасное password hashing.

===== 4. CUSTOMER APP
=====

Главные разделы:

Главная Каталог Поиск Корзина Заказы Избранное Профиль

Главная должна содержать:

- поиск;
- баннеры;
- категории;
- популярные товары;
- новинки;
- скидки;
- рекомендации;
- популярные магазины;
- бренды.

===== 5. CATALOG
=====

Создать:

- категории;
- подкатегории;
- бренды;
- товары;
- варианты товаров;
- характеристики;
- изображения;
- видео;
- SKU.

Категории должны поддерживать неограниченную вложенность.

Характеристики должны быть динамическими.

Например для ноутбука:

CPU RAM SSD GPU Display OS

Для одежды:

Size Color Material

Не создавать характеристики жёстко только под одну категорию.

===== 6. PRODUCT
=====

Карточка товара должна иметь:

- фотографии;
- видео;
- название;
- бренд;
- SKU;
- рейтинг;
- количество отзывов;
- цену;
- старую цену;
- скидку;
- остаток;
- описание;
- характеристики;
- варианты;
- продавца;
- рейтинг продавца;
- доставку;
- возврат.

Добавить:

Add to Cart Buy Now Favorite Share

===== 7. SEARCH
=====

Реализовать:

- поиск по названию;
- SKU;
- бренду;
- категории;
- описанию;
- характеристикам.

Добавить:

- autocomplete;

- typo tolerance;
- pagination;
- sorting;
- filtering.

Подготовить архитектуру для Elasticsearch/OpenSearch.

===== 8. CART
=====

Корзина должна поддерживать товары разных продавцов.

Backend должен автоматически разделять единый customer order на seller orders.

Пример:

Customer Order | |-- Seller Order A | |-- Product 1 | |-- Product 2 | |-- Seller Order B |-- Product 3

===== 9. ORDER
=====

Создать полноценную order state machine.

Статусы:

CREATED PAYMENT_PENDING PAID CONFIRMED PROCESSING PACKED READY_FOR_PICKUP SHIPPED
IN_DELIVERY DELIVERED COMPLETED CANCELLED RETURN_REQUESTED RETURNED REFUNDED

Запрещено менять статус заказа произвольным образом.

Каждый переход должен проверяться backend.

===== 10. PAYMENT
=====

Создать отдельный Payment Module.

Подготовить интеграцию:

- Payme;
- Click;
- банковские карты;
- другие payment providers.

Не хранить card data.

Поддержать:

pending processing paid failed cancelled refunded partially_refunded

Создать webhook endpoints.

Webhook должен быть idempotent.

===== 11. SELLER
=====

Seller Dashboard:

Dashboard Products Inventory Orders Customers Reviews Promotions Advertising Finance Analytics
Settings

Seller должен иметь возможность:

- создать магазин;
- добавить товар;
- изменить товар;
- загрузить изображения;
- указать цену;
- указать остаток;
- создавать варианты;
- получать заказы;
- менять разрешённые статусы;
- смотреть продажи;
- смотреть баланс.

===== 12. MODERATION
=====

Каждый новый товар проходит moderation.

Статусы:

DRAFT PENDING_MODERATION APPROVED REJECTED BLOCKED

Модератор должен видеть:

- название;
- фото;
- описание;
- категорию;
- цену;
- продавца.

Причина отклонения обязательна.

===== 13. REVIEWS
=====

Оставлять отзыв может только пользователь, который реально приобрёл товар.

Review:

- rating;
- title;
- text;
- images;
- orderId;
- productId;
- userId.

Защитить систему от повторных фальшивых отзывов.

===== 14. CHAT
=====

Создать realtime chat.

Customer ↔ Seller.

Поддержать:

- text;
- images;
- timestamps;
- read status;
- notifications;
- report;
- block.

===== 15. DELIVERY
=====

Создать Delivery Module.

Поддержать:

Seller Delivery Marketplace Delivery Pickup Point

Создать:

- couriers;
- delivery zones;
- pickup points;
- tracking;
- delivery statuses.

===== 16. RETURN
=====

Покупатель может создать return request.

Необходимы:

- причина;
- описание;
- фотографии;
- order;
- item.

Статусы:

REQUESTED APPROVED REJECTED IN_RETURN RECEIVED REFUNDED

===== 17. PROMOTIONS
=====

Создать:

- coupons;
- promo codes;
- discounts;
- flash sales;
- free delivery;
- product promotions.

===== 18. ADVERTISING
=====

Создать рекламную систему для продавцов.

Возможности:

- promoted product;
- search promotion;
- banner;
- recommendation placement.

Считать:

- impressions;
- clicks;
- CTR;
- orders;
- revenue.

===== 19. COMMISSION
=====

Marketplace получает комиссию.

Комиссия должна быть configurable.

Можно установить:

global commission category commission seller commission product commission

Расчёт:

sale amount

- marketplace commission

- refunds = seller balance

===== 20. SELLER FINANCE
=====

Создать:

Seller Balance Transactions Payouts Commission Refunds

Payout statuses:

REQUESTED PROCESSING PAID REJECTED

===== 21. ADMIN
=====

Admin Dashboard должен иметь:

Dashboard Users Sellers Stores Products Categories Brands Orders Payments Returns Delivery Couriers
Pickup Points Reviews Complaints Coupons Promotions Advertising Commissions Payouts Analytics
Settings Audit Logs

===== 22. NOTIFICATIONS
=====

Создать Notification Service.

Поддержать:

Push Email SMS Telegram

Архитектура должна позволять легко подключать новые providers.

===== 23. DATABASE
=====

Создать нормализованную PostgreSQL database.

Основные сущности:

users user_profiles roles permissions sellers stores seller_documents categories brands products
product_variants product_images product_attributes inventory inventory_movements carts cart_items
orders order_items seller_orders payments payment_transactions addresses deliveries couriers
pickup_points favorites reviews review_images chats chat_messages notifications promotions coupons
advertising_campaigns commissions seller_balances payouts returns return_items complaints
audit_logs

Использовать:

- UUID;
- foreign keys;
- indexes;
- unique constraints;
- timestamps;
- soft delete там, где необходимо.

===== 24. API
=====

Создать versioned REST API:

/api/v1/...

Все endpoints должны иметь:

- validation;
- authorization;
- DTO;
- pagination;
- filtering;
- sorting;
- error handling.

Создать Swagger/OpenAPI.

===== 25. SECURITY
=====

Обязательно:

- HTTPS;
- JWT;
- refresh tokens;
- RBAC;
- rate limiting;
- input validation;
- sanitization;
- secure headers;
- CORS;
- protection against XSS;
- SQL injection protection;

- CSRF protection;
- brute-force protection;
- audit logs;
- secure secrets management.

Никогда не доверять frontend данным.

Все критические расчёты выполнять на backend.

Цена товара, скидка, комиссия, остаток и сумма заказа должны повторно проверяться backend перед оформлением заказа.

===== 26. MOBILE
=====

Создать Android/iOS приложение.

Экран:

Splash Onboarding Login Home Catalog Search Product Cart Checkout Payment Orders Order Details
Favorites Profile Notifications Chat

Приложение должно поддерживать push notifications.

===== 27. WEB
=====

Создать:

Customer Web Seller Dashboard Admin Dashboard

Web должен быть responsive.

Desktop Tablet Mobile

===== 28. UX/UI
=====

Не копировать интерфейс Wildberries.

Создать собственный дизайн.

Требования:

- modern;
- minimal;
- clean;
- fast;
- mobile-first;
- удобная навигация;

- большие фотографии;
- понятная цена;
- заметные СТА;
- хорошая типографика;
- accessibility.

===== 29. PERFORMANCE =====

Цели:

- быстрый first load;
- lazy loading изображений;
- image optimization;
- CDN;
- caching;
- database indexes;
- pagination;
- API caching;
- background jobs.

Не загружать весь каталог сразу.

===== 30. MONITORING =====

Подготовить:

- logging;
- error tracking;
- metrics;
- health checks;
- database monitoring;
- API monitoring.

Создать:

GET /health

===== 31. TESTING =====

Обязательно создать:

Unit tests Integration tests E2E tests API tests

Критические сценарии:

- регистрация;
- login;
- создание товара;

- moderation;
- cart;
- checkout;
- payment;
- order;
- cancellation;
- return;
- seller payout.

===== 32. DEVOPS =====

Создать Docker environment.

Отдельные:

development staging production

Создать:

docker-compose.yml

Подготовить CI/CD.

Pipeline:

lint test build security check deploy

===== 33. PROJECT STRUCTURE =====

Использовать monorepo:

marketplace/

apps/ web/ mobile/ api/ admin/

packages/ ui/ types/ utils/ config/

infrastructure/ docker/ nginx/

docs/

===== 34. MVP =====

Сначала реализовать рабочий MVP:

1. Auth
2. User

3. Seller
4. Store
5. Categories
6. Products
7. Search
8. Filters
9. Cart
10. Orders
11. Payment abstraction
12. Seller Dashboard
13. Admin Dashboard
14. Reviews
15. Notifications

После завершения MVP реализовать:

Delivery Returns Advertising Promotions Analytics AI features

===== 35. AI FEATURES — ПОЗЖЕ
=====

Архитектура должна быть готова к:

AI product moderation AI search AI recommendations AI chatbot AI seller assistant AI fraud detection
AI product description generation

===== 36. ПОРЯДОК РАЗРАБОТКИ
=====

Не пытайся написать весь проект одним огромным файлом.

Работай итеративно.

ШАГ 1: Architecture

ШАГ 2: Database schema

ШАГ 3: Backend foundation

ШАГ 4: Authentication

ШАГ 5: Catalog

ШАГ 6: Products

ШАГ 7: Search

ШАГ 8: Cart

ШАГ 9: Orders

ШАГ 10: Payment abstraction

ШАГ 11: Seller Dashboard

ШАГ 12: Admin Dashboard

ШАГ 13: Mobile application

ШАГ 14: Notifications

ШАГ 15: Testing

ШАГ 16: Docker

ШАГ 17: Deployment

После каждого этапа проверяй:

- compilation;
- TypeScript errors;
- database migrations;
- API;
- authorization;
- tests;
- integration.

Не переходи к следующему крупному этапу, если предыдущий находится в нерабочем состоянии.

===== 37. КОД
=====

Использовать:

TypeScript strict mode.

Не использовать:

- any без необходимости;
- hardcoded secrets;
- hardcoded prices;
- hardcoded commission;
- дублирование бизнес-логики;
- огромные компоненты;
- огромные файлы;
- fake APIs в production code.

Каждый модуль должен иметь понятную ответственность.

===== 38. ERROR HANDLING
=====

Создать единый формат ошибок API:

```
{ "success": false, "error": { "code": "...", "message": "...", "details": {} } }
```

Ошибки должны быть понятны frontend.

===== 39. DOCUMENTATION
=====

Создать:

README.md

Architecture documentation.

API documentation.

Database documentation.

Deployment documentation.

Environment variables documentation.

Seller documentation.

Admin documentation.

===== 40. ГЛАВНОЕ ПРАВИЛО
=====

Создаваемая система должна быть реальным marketplace-продуктом, а не демонстрационным шаблоном.

Не использовать fake data вместо реально работающей бизнес-логики там, где требуется backend.

Не имитировать оплату, заказ, остатки или баланс продавца в production architecture.

Все критические операции должны выполняться на backend.

Сначала построить качественную архитектуру и MVP, затем постепенно расширять систему.

Не копировать бренд, логотип, тексты или уникальный дизайн Wildberries.

Использовать Wildberries только как ориентир по функциональному классу marketplace.

Конечный результат:

полноценная собственная marketplace-платформа Web + Android + iOS + Seller Dashboard + Admin Dashboard, готовая к дальнейшему масштабированию.

Начни с: 1. архитектуры; 2. структуры monorepo; 3. database schema; 4. backend modules; 5. API contracts; 6. MVP roadmap.

После этого переходи к реализации по этапам.